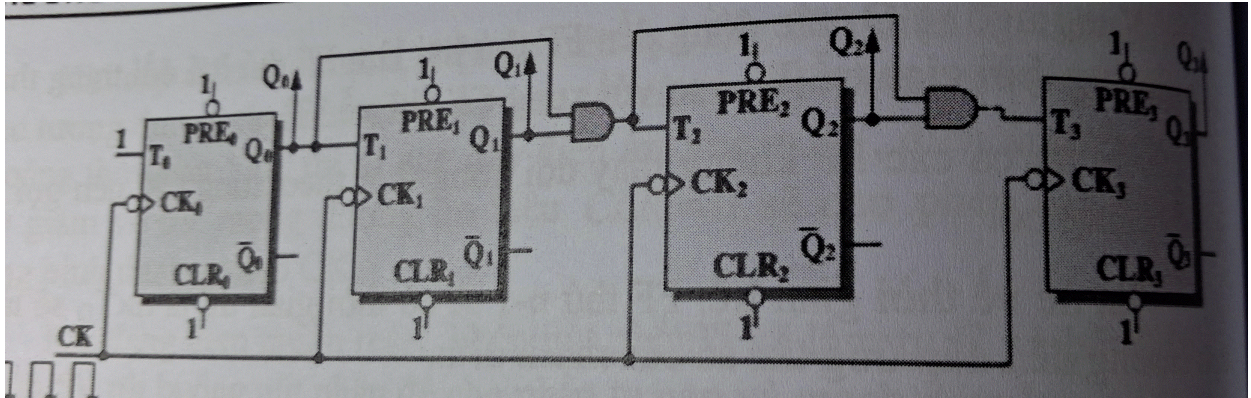


VibeCoding tạo code phân tích Mạch đếm đồng bộ 4-bit.

Prof Happy AI-Sưu tầm và biên soạn với sự hỗ trợ của Gemini, May 5, 2026



Đây là một Mạch đếm đồng bộ 4-bit (4-bit Synchronous Binary Counter) sử dụng các T Flip-Flop. (Trang 210 Sách kỹ thuật số, trường ĐH Sư Phạm Kỹ thuật Tp.HCM, Nguyễn Đình Phú, Nguyễn Trường Duy) Phân tích mạch:

Cấu trúc: Gồm 4 Flip-Flop T_0, T_1, T_2, T_3 .

Tín hiệu Clock (CK): Tất cả các Flip-Flop đều dùng chung một nguồn xung clock (kích hoạt cạnh xuống - negative edge, dựa trên vòng tròn ở chân CK), đặc trưng của mạch đồng bộ.

Nguyên lý ngõ vào $T: T_0 = 1$ (Luôn luôn lật trạng thái mỗi khi có xung clock).

$T_1 = Q_0$. $T_2 = Q_0 \cdot Q_1$ (Sử dụng cổng AND). $T_3 = (Q_0 \cdot Q_1) \cdot Q_2$ (Sử dụng cổng AND nối tiếp).

Kết quả: Mạch này sẽ đếm nhị phân từ 0000 đến 1111 (0 đến 15 thập phân).

Mã Python mô phỏng: Dưới đây là đoạn code mô phỏng hoạt động của mạch qua 16 chu kỳ xung clock.

```
class T_FlipFlop:
    def __init__(self, name):
        self.name = name
        self.Q = 0 # Trạng thái ban đầu

    def update(self, T, clock_edge):
        # Giả định clock_edge = True đại diện cho cạnh xuống (falling edge)
        if clock_edge:
            if T == 1:
                self.Q = 1 - self.Q
```

```

        self.Q = 1 - self.Q # Lật trạng thái (Toggle)
    return self.Q

def simulate_counter(steps=16):
    # Khởi tạo 4 Flip-Flop
    ff0 = T_FlipFlop("FF0")
    ff1 = T_FlipFlop("FF1")
    ff2 = T_FlipFlop("FF2")
    ff3 = T_FlipFlop("FF3")

    print(f"{'Clock':<6} | {'Q3'} {'Q2'} {'Q1'} {'Q0'} | Dec")
    print("-" * 30)

    for i in range(steps + 1):
        # Hiển thị trạng thái hiện tại trước khi có xung clock tiếp theo
        binary_str = f"{ff3.Q}{ff2.Q}{ff1.Q}{ff0.Q}"
        decimal_val = int(binary_str, 2)
        print(f"{i:<6} | {ff3.Q} {ff2.Q} {ff1.Q} {ff0.Q} | {decimal_val}")

        if i == steps: break

        # Tính toán giá trị đầu vào T cho chu kỳ tiếp theo dựa trên Q hiện tại
        t0 = 1
        t1 = ff0.Q
        t2 = ff0.Q & ff1.Q
        t3 = (ff0.Q & ff1.Q) & ff2.Q

        # Cập nhật đồng thời tất cả Flip-Flop tại cạnh xung clock
        ff0.update(t0, True)
        ff1.update(t1, True)
        ff2.update(t2, True)
        ff3.update(t3, True)

if __name__ == "__main__":
    simulate_counter()

```

Clock	Q3	Q2	Q1	Q0	Dec
0	0	0	0	0	0
1	0	0	0	1	1
2	0	0	1	0	2
3	0	0	1	1	3
4	0	1	0	0	4
5	0	1	0	1	5
6	0	1	1	0	6
7	0	1	1	1	7
8	1	0	0	0	8
9	1	0	0	1	9
10	1	0	1	0	10
11	1	0	1	1	11
12	1	1	0	0	12
13	1	1	0	1	13
14	1	1	1	0	14
15	1	1	1	1	15
16	0	0	0	0	0

Để tạo dạng sóng (waveform) cho các tín hiệu của mạch đếm đồng bộ này, chúng ta sẽ sử dụng thư viện matplotlib.

Dựa trên hình ảnh image_2bbeb7.jpg, mạch có các đặc điểm logic sau:

Clock (CK): Kích hoạt cạnh xuống (falling edge).

T 0: Luôn bằng 1.

T 1: Nối với Q 0.

T 2: Nối với đầu ra cổng AND của (Q 0·Q 1).

T 3: Nối với đầu ra cổng AND của (T 2·Q 2).

Dưới đây là mã Python để mô phỏng và vẽ dạng sóng cho đến xung Clock thứ 16 (để thấy trọn chu kỳ $2^4 = 16$).

```
import matplotlib.pyplot as plt
import numpy as np

def simulate_waveforms(periods=16):
    # Thiết lập dữ liệu thời gian (tạo các điểm để vẽ dạng sóng vuông)
    # Mỗi chu kỳ clock có 2 trạng thái: Cao (1) và Thấp (0)
    time = np.arange(0, periods + 0.5, 0.5)

    # Khởi tạo các mảng tín hiệu
    ck = []
    q0, q1, q2, q3 = [0], [0], [0], [0] # Trạng thái ban đầu

    # Biến nội tại của Flip-Flops
    curr_q0, curr_q1, curr_q2, curr_q3 = 0, 0, 0, 0

    for t in range(periods):
        # 1. Tạo xung Clock (Cạnh xuống tại mỗi số nguyên t+1)
        ck.extend([1, 0])

        # 2. Logic điều khiển T (Dựa trên trạng thái hiện tại trước khi Clock)
        t0 = 1
        t1 = curr_q0
        t2 = curr_q0 & curr_q1
        t3 = (curr_q0 & curr_q1) & curr_q2

        # 3. Cập nhật trạng thái Flip-Flop (Tác động cạnh xuống)
        if t0 == 1: curr_q0 = 1 - curr_q0
        if t1 == 1: curr_q1 = 1 - curr_q1
        if t2 == 1: curr_q2 = 1 - curr_q2
        if t3 == 1: curr_q3 = 1 - curr_q3

    # Lưu trữ giá trị để vẽ (mỗi trạng thái lặp lại 2 lần để tạo đường n
    for _ in range(2):
        q0.append(curr_q0)
        q1.append(curr_q1)
        q2.append(curr_q2)
```

```
q3.append(curr_q3)

# Thêm giá trị cuối cho CK để khớp độ dài
ck.append(1)

# Vẽ biểu đồ
signals = [ck, q0, q1, q2, q3]
labels = ['Clock', 'Q0 (LSB)', 'Q1', 'Q2', 'Q3 (MSB)']

fig, axes = plt.subplots(len(signals), 1, figsize=(12, 8), sharex=True)
plt.subplots_adjust(hspace=0.3)

for i, (sig, label) in enumerate(zip(signals, labels)):
    # Vẽ dạng sóng bậc thang (steps-post)
    axes[i].step(time, sig, where='post', color='blue', linewidth=2)
    axes[i].set_ylabel(label, rotation=0, labelpad=30, fontweight='bold')
    axes[i].set_ylim(-0.2, 1.2)
    axes[i].set_yticks([0, 1])
    axes[i].grid(True, alpha=0.3)

plt.xlabel('Clock Cycles')
plt.suptitle('Dạng sóng mạch đếm đồng bộ 4-bit (Negative Edge Triggered)')
plt.xticks(range(periods + 1))
plt.show()

if __name__ == "__main__":
    simulate_waveforms(16)
```

